

# Causal Analysis and Interactive Reasoning over Conversational Data

---

## 1. Problem Statement

Large-scale conversational systems generate vast collections of multi-turn agent–customer dialogues. Some of these conversations are associated with costly outcome events such as escalations, complaints, or refunds. While such outcomes are typically recorded, the causal conversational factors leading to them remain unclear. Existing systems do not identify which dialogue turns contributed to an outcome or how conversational patterns evolved prior to the event.

The challenge is to design a system that can analyze structured conversational transcripts and produce causally grounded, evidence-backed explanations linking conversational behavior to observed outcomes. In addition, users must be able to interactively explore these explanations through follow-up questions, requiring the system to maintain contextual consistency and causal coherence across multiple turns.

The overall objective is to move beyond simple event detection toward interpretable causal analysis and interactive reasoning over conversational data.

---

## 2. Objectives

### Task 1: Query-Driven Causal Explanation with Evidence

Given a natural-language analytical query, the system must:

- Analyze conversations associated with a specific outcome,
- Identify dialogue-level causal factors,
- Extract supporting dialogue spans as evidence,
- Produce a structured and interpretable causal explanation.

### Task 2: Multi-Turn Context-Aware Interaction

The system must:

- Support analytical follow-up queries,
- Retain context from prior interactions,
- Maintain causal and evidential consistency,
- Ensure deterministic behavior across turns.

---

## 3. Core Data Models (Schemas)

All data is governed by strict Pydantic schemas to ensure determinism and traceability:

1. Conversation – normalized transcript with outcome label
2. Turn – individual dialogue turn with speaker role
3. EvidenceSpan – traceable evidence (call\_id, turn\_id, text)
4. CausalFactor – interpretable factor with linked evidence
5. CausalExplanation – structured Task-1 output
6. ContextState – explicit pinned state for Task-2

### **Guarantees**

- No schema drift
  - No hallucinated evidence
  - Fully reproducible outputs
- 

## **4. Approach (Structured System Design)**

### **4.1 Design Philosophy**

The system is built on deterministic causal reasoning rather than predictive modeling or generative language models. The primary focus is interpretability, faithfulness, reproducibility, and contextual consistency.

---

### **4.2 Data Normalization and Validation**

Raw conversational transcripts are converted into a normalized format where each conversation contains: - A unique call ID, - A predefined outcome label, - Ordered dialogue turns with explicit speaker roles.

Strict schema validation eliminates malformed inputs and ensures deterministic processing.

**Output:** Schema-safe normalized conversational data.

---

### **4.3 Task 1: Query-Driven Causal Explanation**

**Objective:** Explain why a specific outcome event occurred.

**Pipeline:** 1. Outcome-Based Selection – select conversations by outcome label  
2. Dialogue-Level Feature Analysis – detect patterns such as frustration or repeated unresolved issues  
3. Causal Factor Identification – group patterns into interpretable causal factors  
4. Evidence Extraction – extract verbatim dialogue turns as evidence  
5. Causal Chain Construction – temporally order causal factors  
6. Structured Explanation Generation – produce a traceable explanation

**Output:** Interpretable causal explanation grounded in verbatim evidence.

---

#### 4.4 Context State Construction

After Task 1, the system initializes a ContextState that: - Pins the active outcome, - Stores relevant call IDs, - Stores extracted evidence spans, - Preserves the causal explanation.

This serves as deterministic memory for Task 2.

---

#### 4.5 Task 2: Multi-Turn Context-Aware Reasoning

**Objective:** Support follow-up queries without losing causal consistency.

**Process:** - Interpret follow-up intent (factors, evidence, summary), - Answer queries using only pinned ContextState, - Prevent evidence drift or hallucination.

**Result:** Stable, context-aware multi-turn interaction.

---

### 5. Evaluation Strategy

The system is evaluated using deterministic metrics aligned with task objectives.

#### 5.1 IDRecall (Evidence Accuracy)

$$\text{IDRecall} = \frac{|\text{Retrieved Call IDs} \cap \text{Ground Truth Call IDs}|}{|\text{Ground Truth Call IDs}|}$$

- Measures exact Call-ID matching
  - Deterministic and reproducible
-

## 5.2 Faithfulness (Hallucination Control)

Faithfulness = (Evidence spans found verbatim in transcript) / (Total evidence spans)

- Verbatim matching by call\_id, turn\_id, speaker, and text
  - No paraphrasing or generation
- 

## 5.3 Relevancy (Conversational Coherence)

- Continuous score in range  $[0, 1]$
  - Based on coverage of expected intent elements
  - $0.5 \rightarrow$  Relevant,  $0.5 \rightarrow$  Not relevant
- 

## 6. Why This Design Works Best

Why NOT RAG / Vector Search / LLMs

| Aspect            | RAG / Vector Search | This System   |
|-------------------|---------------------|---------------|
| Evidence accuracy | Approximate         | Exact         |
| Faithfulness      | Hallucination risk  | Guaranteed    |
| Determinism       | Prompt-dependent    | Deterministic |
| IDRecall          | $< 1.0$             | 1.0           |
| Interpretability  | Low                 | High          |

### Key Insight:

The task is causal explanation, not semantic discovery. Since outcome labels are already known, deterministic evidence selection is optimal.

---

## 7. Results

| Metric       | Value |
|--------------|-------|
| IDRecall     | 1.00  |
| Faithfulness | 1.00  |
| Relevancy    | 0.95  |

---

## **8. Execution Process**

### **Step 1: Create Virtual Environment**

```
“bash python -m venv venv
```

### **Step 2: Install Dependencies**

```
pip install -r requirements.txt
```

### **Step 3: Normalize Dataset**

```
python data/preprocess.py
```

### **Step 4: Run End-to-End Pipeline**

```
python -m scripts.run_demo
```

### **Step 5: Run Notebook**

```
jupyter notebook notebooks/final_pipeline.ipynb
```

### **Step 6: Generate Query Dataset**

```
python scripts/generate_queries.py
```

### **Step 7: Generate PDF**

```
pandoc README.md -o README.pdf --pdf-engine=xelatex
```